Tag — A Multi-Agent Reinforcement Learning Laboratory

Project Proposal

RL Tag Project Team

April 2026

1. Project Overview

This project investigates how different reinforcement learning algorithms learn asymmetric pursuit–evasion behavior in a multi-agent **Tag** environment. We implement a 2D tile-based game in which one **tagger** chases multiple **runners** across a map with walls and movable obstacles, and train agents to play both roles through self-play.

Tag is a minimal but information-rich setting where emergent strategies — pursuit, evasion, cornering, and obstacle use — are directly observable. The role asymmetry further lets us study how each algorithm handles role-specific credit assignment within a single shared training loop.

2. Research Objectives

1. **Algorithm comparison.** Train and benchmark a suite of RL algorithms — Q-Learning, SARSA, DQN, PPO, and DPO — on the same environment, and compare their sample efficiency, final performance, and behavioral quality.
2. **Role-specific learning.** Analyze how well each algorithm masters the two roles (tagger vs. runner) *independently*, using isolated role evaluation that pits a trained role against a random opponent.
3. **Reward sensitivity.** Study how reward design choices affect learned behavior, and identify pathological patterns (corner-camping, standing still, reward-farming).

3. Technical Approach

3.1 Environment

- 2D grid-based Tag game built on Pygame with walls, spawn points, and movable crates.
- 6 agents per simulation on the default map (one tagger and five runners at any time), with tag events transferring the tagger role. Larger maps support more agents.
- Parallel headless simulations for fast CPU-based training; optional Pygame display mode for live behavioral inspection.

3.2 Observation (ego-centric)

Each agent observes a normalized, self-centered view of the world:

- Own position and velocity.

- Relative position and distance of every other active agent, with the tagger and nearest runner explicitly highlighted.
- 8-direction wall raycasts (normalized distance to the nearest wall along each ray).
- Relative positions of nearby movable crates.
- Eliminated agents are filtered out so that policies do not perceive or respond to inactive opponents.

3.3 Reward Design (Pure-Proximity, "Plan C")

After iterating through several earlier designs, we converged on a minimal, interpretable scheme:

- **Terminal signals dominate:** `+50` on a successful tag, `-80` on being tagged. The asymmetry pushes runners toward caution and taggers toward aggression.
- **Quadratic proximity field** within a fixed 140-pixel radius: the tagger earns a bonus and the runner takes a symmetric penalty, with the sharpest gradient located near the tag zone.
- **Constant time pressure** (`-0.05/step` for the tagger) and **constant survival bonus** (`+0.10/step` for the runner), with no time-scale amplifier.
- Long-range navigation is delegated to the observation (via `nearest_runner_rel` / `tagger_rel`); the reward stays silent outside the proximity radius, forcing the policy to rely on perception rather than reward gradients for coarse movement.

3.4 Dual-Role Architecture

Agents share **two** networks — one *tagger brain* and one *runner brain* — selected at runtime based on the agent's current role. All six agents contribute experience to the same shared networks, dramatically improving sample efficiency while preserving role-specific specialization.

3.5 Training and Evaluation Pipeline

A unified `run_experiment.py` pipeline handles the full workflow for any algorithm:

1. **Train** with periodic checkpoints at configurable epoch milestones, plus a *best-so-far* checkpoint that captures the true policy peak — important for algorithms such as PPO that oscillate round-to-round.
2. **Joint evaluation** of each checkpoint, with the policy frozen at the algorithm level (preventing stale transitions from polluting the network during eval) and a fixed random seed (ensuring fair cross-checkpoint comparison).
3. **Isolated role evaluation:** trained tagger vs. untrained runners (more tags = better tagger) and trained runners vs. untrained tagger (fewer tags = better runners).
4. **Plotting:** training curves, per-epoch performance bars, role-trend line plots, role diagnostics (advantage index, stability), trajectories, and position heatmaps.

4. Expected Deliverables

- A reproducible RL pipeline supporting headless CPU training and optional live visualization.
- Trained checkpoints and evaluation metrics for each algorithm at multiple training epochs.
- A comparative report analyzing algorithm performance, role-specific behavior, and reward sensitivity.
- Trajectory visualizations and diagnostic plots for qualitative behavioral analysis.

5. Team Responsibilities

Each team member owns one algorithm's implementation and experiments, while sharing a common environment, pipeline, and evaluation protocol so results are directly comparable:

Member	Algorithm	Category
—	Q-Learning	Tabular baseline
—	SARSA	Tabular baseline
—	DQN	Value-based deep RL
—	PPO	On-policy policy gradient
(author)	DPO	Direct preference optimization

6. Timeline (tentative)

Phase	Duration	Milestones
Setup & environment	Week 1–2	Game engine, observation/reward design, training pipeline
Baseline training	Week 3–4	Q-Learning / SARSA / DQN checkpoints and initial evaluation
Advanced methods	Week 5–6	PPO and DPO training, hyperparameter tuning
Comparison & analysis	Week 7	Cross-algorithm evaluation, plots, behavioral analysis
Report & presentation	Week 8	Final report and presentation